

Agent Hook 系统对比

Pi 三层 Hook vs LangChain Callback

技术架构白皮书 · 2026年6月

执行摘要

Pi 和 LangChain 采用截然不同的 Agent 事件系统设计: Pi 使用三层 Hook 架构 (Agent Loop + Agent Event + Harness/Extension), 支持 block/patch/cancel 语义; LangChain 使用 Mixin-Based 的 Callback 系统, 纯只读观测。本文档从架构设计、ReAct Loop 映射、修改能力、扩展性四个维度进行系统性对比, 揭示两种设计哲学背后的权衡与适用场景。

一、架构设计对比

1.1 核心架构

维度	Pi	LangChain
架构模式	三层分层 Hook	Mixin-Based 回调

核心抽象	Agent Loop 回调 + AgentEvent 流 + HookEvent 拦截	BaseCallbackHandler (Mixin 继承链)
类型系统	TypeScript + Phantom Type (HookResult)	Python Mixin + 鸭子类型
执行模型	同步/异步混合, 返回值参与控制流	异步聚合, 回调并行执行
错误处理	Hook 可 block, 错误可传播	错误隔离, 一个 handler 崩溃不影响其他

1.2 层级对应关系

关键发现: LangChain 的 Callback 系统 只对应 Pi 的 **Agent** 事件层(只读观测), 没有 Agent Loop 层和 Harness/Extension 层的等价物。

Pi 层级	能力	LangChain 等价物	差距
Agent Loop 层	函数回调, 直接参与执行逻辑 可 block 工具、修改消息、决定停止	无	LangChain 没有执行控制点
Agent 事件层	只读事件流, 状态同步、UI 更新	BaseCallbackHandler	等价, 但 Pi 的事件粒度更细
Harness/Extension 层	类型安全拦截, block/patch/cancel	无	LangChain 无法拦截执行

二、ReAct Loop 事件映射对比

2.1 事件触发时机对比

ReAct 阶段	Pi 事件	LangChain 事件
整轮开始	agent_start , before_agent_start	on_chain_start

LLM 请求	<code>message_start</code> , <code>message_update</code> × N, <code>message_end</code>	<code>on_llm_start</code> , <code>on_llm_new_token</code> × N, <code>on_llm_end</code>
工具调用	<code>beforeToolCall</code> , <code>tool_execution_start/end</code> , <code>afterToolCall</code>	<code>on_agent_action</code> , <code>on_tool_start</code> , <code>on_tool_end</code>
Agent 完成	<code>agent_end</code> , <code>turn_end</code>	<code>on_agent_finish</code> , <code>on_chain_end</code>

2.2 关键差异

Pi 优势：

- `message_start` 区分 user/assistant/toolResult 三种消息类型
- `turn_start/end` 明确标记单轮边界
- `save_point` 标记状态固化点
- 流式事件包含完整消息对象，不只是 token 字符串

LangChain 优势：

- `on_llm_new_token` 更细粒度的流式输出
- `on_retry` 专门的 retry 事件
- `on_custom_event` 支持自定义事件扩展
- Mixin 设计可组合，按需继承

三、修改能力对比

3.1 核心能力矩阵

操作	Pi	LangChain
修改 System Prompt	✓ <code>before_agent_start</code> 返回新 systemPrompt	✗ 无法修改
修改进入 LLM 的消息	✓ <code>context</code> / <code>transformContext</code> 替换 messages	✗ 无法修改
阻断工具调用	✓ <code>beforeToolCall</code> / <code>tool_call</code> 返回 { block: true }	✗ 无法阻断
修改工具结果	✓ <code>afterToolCall</code> / <code>tool_result</code> patch 结果	✗ 无法修改

修改 LLM 请求参数	✔ before_provider_request 修改 model/streamOptions	✘ 无法修改
修改流式 Token	⚠ 不支持(设计上 AssistantMessage 不可修改)	✘ 不支持
决定循环终止	✔ shouldStopAfterTurn 返回 boolean	✘ 无法控制
观测执行流程	✔ Agent 事件层 observe()	✔ 所有回调都是观测

3.2 设计哲学差异

Pi: "执行前/中/后的拦截与修改" —— 精细化控制 Agent 行为
LangChain: "执行后的观测与记录" —— 追踪 Agent 行为

四、扩展性对比

4.1 Extension / 自定义 Handler

维度	Pi Extension	LangChain Custom Handler
注册方式	文件系统扫描 extensions/ 目录	代码中实例化并传入 callbacks=[...]
生命周期	随 Session 持久化, 重启后自动加载	随代码实例化, 每次调用独立
注册工具	✔ pi.registerTool()	✔ 通过 tools=[...] 传入
注册命令	✔ pi.registerCommand()	✘ 无内置命令系统
UI 交互	✔ ctx.ui.select/confirm/input	✘ 无 UI 系统
类型安全	Phantom Type + 泛型约束	Python 鸭子类型
错误隔离	Extension 错误不破坏核心	Handler 错误不破坏其他 handler

4.2 沙盒性对比

Pi Extension 沙盒:

- 只能通过 Harness HookEvent 参与生命周期

- 无法直接操作 Agent 类实例
- 所有修改通过声明式结果对象传递
- Session 持久化保证可审计性

LangChain Handler 自由度：

- 可以在回调中执行任意 Python 代码
- 可以发起网络请求、读写文件
- 但不能修改执行状态
- 每次调用独立，无持久化状态

五、适用场景对比

5.1 选择 Pi 的场景

- 需要拦截执行：阻断危险工具调用、修改工具结果
- 需要修改上下文：动态裁剪消息历史、注入系统指令
- 需要精细化控制：切换模型、改变思考级别、控制循环终止
- 需要可恢复性：Session 持久化、崩溃后恢复状态
- 需要 UI 扩展：自定义命令、快捷键、对话框

5.2 选择 LangChain 的场景

- 需要观测追踪：记录 Token 用量、延迟指标
- 需要流式推送：WebSocket、SSE 实时推送
- 需要集成外部系统：LangSmith、Langfuse、自定义日志
- 需要快速原型：简单的回调即可满足需求
- 需要与 Python 生态集成：DataDog、Prometheus 等

六、一句话总结

Pi = 拦截与修改框架：告诉你“发生了什么”，还能决定“接下来发生什么”。

LangChain Callback = 观测与追踪框架：告诉你“发生了什么”，但无法影响“接下来发生什么”。

两者不是竞争关系，而是互补：**LangChain** 适合观测和追踪，**Pi** 适合控制和扩展。在实际项目中，可以根据需求选择或组合使用。